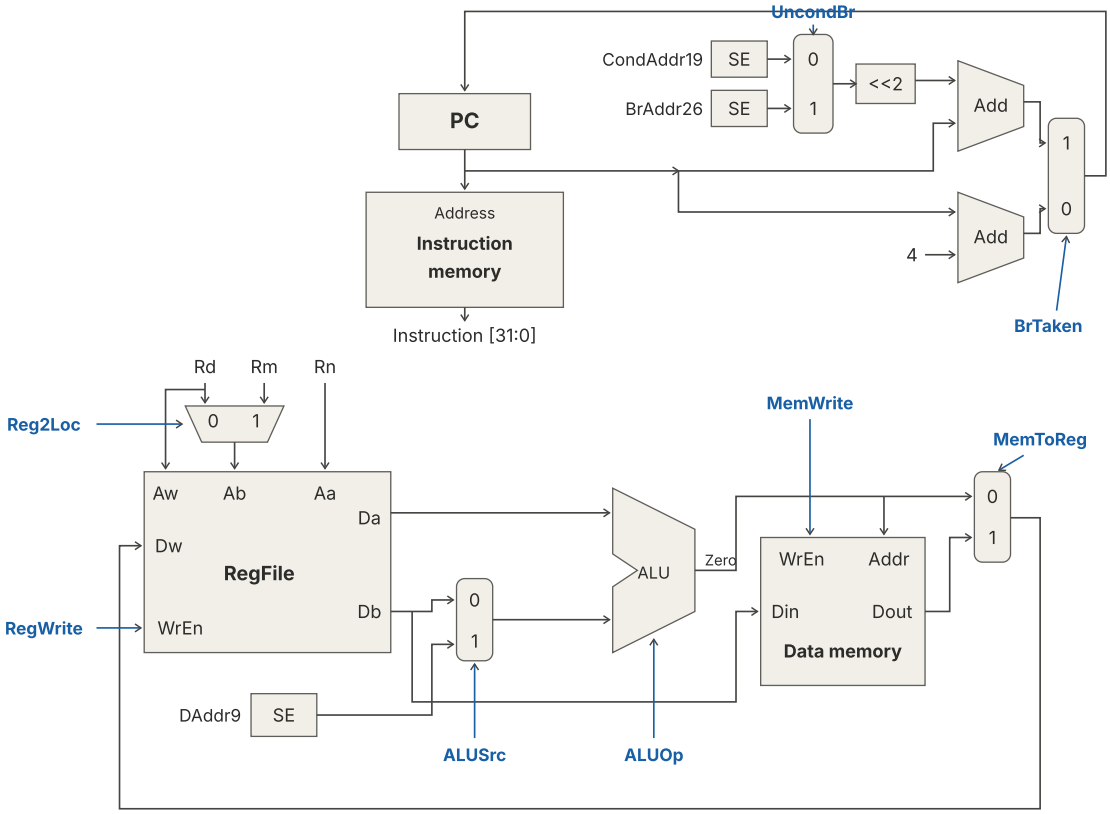
EE469 single-cycle datapath: label key

Every label on the class datapath (slide 87), what it means, and its assembly equivalent. Includes the labels added in HW3 problem 9.

1. The labeled datapath



Black = hardware and data wires. Blue = control signals set by the control unit for each instruction.

2. Instruction formats (which bits hold which field)

R-type (ADD, SUB, LW_2reg, SWAP, WAI)

Opcode [31:21]	Rm [20:16]	shamt [15:10]	Rn [9:5]	Rd [4:0]

D-type (LDUR, STUR, BRMI)

Opcode [31:21]	DAddr9 [20:12]	00 [11:10]	Rn [9:5]	Rd [4:0]

CB-type (CBZ, LIU)

Opcode [31:24]	CondAddr19 / imm19 [23:5]	Rd [4:0]

B-type (B)

Opcode [31:26]	BrAddr26 [25:0]

3. Instruction fields and where they appear in assembly

Label	Bits	Meaning	Where it is in assembly	Example
Rd	[4:0]	Destination register (for STUR and CBZ it is read, not written)	1st register	`ADD X1, X2, X3`
Rn	[9:5]	First source register / base address	2nd register	`ADD X1, X2, X3`
Rm	[20:16]	Second source register	3rd register	`ADD X1, X2, X3`
DAddr9	[20:12]	Signed byte offset for loads and stores	# number inside [ ]	`LDUR X1, [X2, #16]`
CondAddr19	[23:5]	Signed CBZ offset, counted in instructions	label after the register	`CBZ X1, LOOP`
BrAddr26	[25:0]	Signed B offset, counted in instructions	label	`B LOOP`
imm19	[23:5]	Unsigned constant for LIU	# number	`LIU X5, #100`
Opcode	top bits	Says which instruction this is; the control unit decodes it	the mnemonic	`ADD X1, X2, X3`

4. Instructions: assembly to pseudocode (RTL)

Instruction	Assembly example	Pseudocode	In plain words
ADD	`ADD X1, X2, X3`	`Reg[Rd] = Reg[Rn] + Reg[Rm]`	X1 = X2 + X3
SUB	`SUB X1, X2, X3`	`Reg[Rd] = Reg[Rn] - Reg[Rm]`	X1 = X2 - X3
LDUR	`LDUR X1, [X2, #16]`	`Reg[Rd] = Mem[Reg[Rn] + SE(DAddr9)]`	X1 = memory at X2 + 16
STUR	`STUR X1, [X2, #16]`	`Mem[Reg[Rn] + SE(DAddr9)] = Reg[Rd]`	memory at X2 + 16 = X1
B	`B LOOP`	`PC = PC + SE(BrAddr26) << 2`	always jump
CBZ	`CBZ X1, LOOP`	`if (Reg[Rd]==0) PC = PC + SE(CondAddr19) << 2`	jump only if X1 is 0
HW3 problem 9 instructions			
LIU	`LIU X5, #100`	`Reg[Rd] = ZE(imm19)`	X5 = 100, top bits all 0
LW_2reg	`LW_2reg X1, [X2, X3]`	`Reg[Rd] = Mem[Reg[Rn] + Reg[Rm]]`	X1 = memory at X2 + X3
SWAP	`SWAP X1, X2`	`Reg[Rd] = Reg[Rn]; Reg[Rn] = Reg[Rd]`	X1 and X2 trade values
BRMI	`BRMI [X2, #16]`	`PC = Mem[Reg[Rn] + SE(DAddr9)]`	jump to the address stored at X2 + 16
WAI	`WAI X1`	`Reg[Rd] = PC`	X1 = address of this instruction

Every instruction also starts with `Instruction = Mem[PC]`, and every non-branch ends with `PC = PC + 4`.

5. Hardware labels

Label	What it is / does	Used by
PC	Program counter: holds the address of the current instruction	all
Instruction memory	Holds the program; give it the PC and it returns the 32-bit instruction	all
Add with 4	Computes PC + 4, the next instruction (each instruction is 4 bytes)	all except taken branches
SE (sign-extend)	Widens a short field to 64 bits by copying its top bit, so negatives stay negative	LDUR, STUR, B, CBZ, BRMI
<<2	Shift left 2 = multiply by 4; turns an instruction count into a byte offset	B, CBZ
Branch adder	Computes PC + offset, the branch target	B, CBZ
RegFile	32 registers, X0 to X31, each 64 bits	all except B
Aa / **Da**	Read port a: register number (always Rn) / value read out	ADD, SUB, LDUR, STUR, LW_2reg, SWAP, BRMI
Ab / **Db**	Read port b: register number (Rm or Rd) / value read out	ADD, SUB, STUR, CBZ, LW_2reg, SWAP
Aw / **Dw** / **WrEn**	Write port: register to write (always Rd) / value to write / write permission	ADD, SUB, LDUR, LIU, LW_2reg, SWAP, WAI
ALU	Arithmetic logic unit: add, subtract, pass B, etc.	most
Zero	ALU flag: 1 when the result is 0	CBZ
Data memory	Holds program data (arrays, variables)	LDUR, STUR, LW_2reg, BRMI
Addr / **Din** / **Dout**	Memory address / data to store / data read out	loads and stores

6. Control signals (blue labels)

Signal	Controls	0 means	1 means	Example of 1
Reg2Loc	mux in front of Ab	Rd to Ab	Rm to Ab	`ADD X1, X2, X3`
ALUSrc	mux in front of ALU input B	Db (register)	SE(DAddr9)	`LDUR X1, [X2, #16]`
ALUOp	the ALU operation	add, subtract, pass B, ...		`SUB = subtract`
MemToReg	mux in front of Dw	ALU result	Dout (memory)	`LDUR X1, [X2, #16]`
RegWrite	register file WrEn	no write	write Reg[Rd]	`ADD X1, X2, X3`
MemWrite	data memory WrEn	no write	write memory	`STUR X1, [X2, #16]`
BrTaken	mux in front of PC	PC + 4	branch target	`B LOOP`
UncondBr	mux choosing the branch offset	CondAddr19 (CBZ)	BrAddr26 (B)	`B LOOP`
X	Don't care: the value cannot affect the result. Write enables (RegWrite, MemWrite) are never X.			

7. Labels added in HW3 problem 9

Label	What it is	Machine
ZE (zero-extend)	Widens imm19 to 64 bits by filling bits [63:19] with 0. Wires only, no gates.	LIU
ALUSrc = 2 (binary 10)	New 3rd ALUSrc mux input that selects ZE(imm19). ALUSrc becomes 2 bits wide.	LIU
ImmZE (option 2)	Alternative to a 3:1 mux: a second 2:1 mux after ALUSrc; 1 selects ZE(imm19).	LIU
pass B	ALU operation that copies input B straight to the output	LIU (also CBZ)
Port 2: Aw2, Dw2	Second register-file write port: address Aw2 = Rn, data Dw2 = Db (Reg[Rd])	SWAP
RegWrite2	Write enable for port 2; 1 only for SWAP	SWAP
MemToReg = 2	New 3rd MemToReg input: Da for SWAP, or the PC for WAI	SWAP, WAI
BrMem	New mux in front of the PC; 1 loads the PC from data memory Dout	BRMI

> Each HW3 machine is separate: the full slide 87 datapath plus only one instruction's additions. Each machine's control table lists the 6 class instructions plus that one new instruction.